VETRA

Admin & Operations Guide

Updated October 2026

This is the document for whoever **operates** VETRA day to day, rather than builds it. `BACKEND_GUIDE.md` covers the API and the data model, `backend/README.md` covers running and deploying the code, `DOCUMENTATION.md` covers what each page does, and `DELIVERY_SYSTEM.md` and `PAYMENTS.md` go deep on deliveries and money. This one covers the live services VETRA runs on, how the database is put together and why, how to reach it directly when the admin console itself cannot help (locked out, no Super Admin left, need to look something up by hand), and how to create a Super Admin without ever opening the console.

It is a runbook, not a feature list. Keep it updated whenever a service, credential, or piece of the schema changes in a way that would make a page of this wrong.

---

1. The services VETRA runs on

Each of these is a separate account on a separate site with its own login. There is no single dashboard showing all of them.

Service	What it does for VETRA	Where you manage it
GitHub	Holds the source (`HuddlesTechnologies/VETRA`). cPanel pulls from it; pushing alone does not deploy anything.	github.com, with an account that has access to the `HuddlesTechnologies` org.
cPanel hosting (Namecheap)	Everything that runs: the website (public pages plus the customer, vendor, rider and admin apps) at `https://vetrahq.org` , the API at `https://api.vetrahq.org` (Setup Node.js App, Phusion Passenger, app root `~/vetra-api`), the MySQL database, the delivery sweep cron job, and the `noreply@vetrahq.org` mailbox.	cPanel: Git Version Control (deploys), Setup Node.js App (restart, Node version), MySQL Databases / phpMyAdmin, Cron Jobs, File Manager (`~/vetra-api/.env` , logs).
Render / Vercel / Clever Cloud	The previous API host, frontend host and database. **Rollback only**, kept for about a week after the cPanel cutover and then deleted (`backend/README.md` , "Moving to cPanel", step 11). The old Vercel site still talks to Render and the old database, so anything written there is not on the live site.	dashboard.render.com (`vetra-api`), vercel.com (`vetra-vercel`), console.clever-cloud.com.
Cloudinary	Every uploaded file: avatars, cover photos, product images and video, site banners, KYC documents. Nothing is ever written to local disk.	cloudinary.com dashboard.

Service	What it does for VETRA	Where you manage it
cPanel mail (SMTP)	Every transactional email: password resets, admin invites, rider invites, 2FA codes, order and delivery updates, KYC decisions, account status changes, low-stock alerts. Sent from `noreply@vetrahq.org`.	cPanel → Email Accounts (the mailbox), Email Deliverability (SPF/DKIM), Track Delivery (per-message log).
Anthropic	The AI shopping assistant's replies (`POST /api/assistant/chat`).	console.anthropic.com.
Paystack	All money: buyers pay at checkout, riders and vendors are paid by transfer from the Paystack balance, refunds go back to buyers' cards. Also the bank list and account-name lookup for payout accounts, and the balance card Super Admins see. See `PAYMENTS.md`.	dashboard.paystack.com: Transactions, Transfers, Balance, Settings → API Keys & Webhooks (webhook URL), Settings → Preferences (transfer OTP must be off).
IDCheck.ng	Identity verification for vendor KYC: NIN, BVN and driver's licence checks against the vendor's legal name, and CAC registration lookups. Paid per lookup from a prepaid wallet. Replaced CheckID.ng in October 2026.	idcheck.ng dashboard (applications and API keys, wallet top-up, per-call history).
Smartsupp	A third-party live-chat widget embedded on the public site and the customer and vendor apps (not the admin console). Support chat with VETRA, not part of the app's own data, and not buyer-to-vendor messaging, which does not exist.	smartsupp.com dashboard.

How a change actually goes live

This trips people up, so it is worth stating plainly.

One deploy covers the website and the API together, and it is **manual**:

1. Push to `main` on GitHub.
2. cPanel → **Git Version Control** → Manage → Pull or Deploy → **Update from Remote**.
3. Then **Deploy HEAD Commit**.

That runs `.cpanel.yml`: it copies the website into `~/vetrahq.org` (markdown, PDFs, `backend/` and tooling files are excluded), copies `backend/` into `~/vetra-api` without touching `.env`, `node_modules` or `tmp/`, runs `npm install` and `npm run migrate` inside the app's Node virtualenv, and touches `tmp/restart.txt` so Passenger restarts the API on its next request. A new migration file therefore applies itself on that same deploy.

Skipping step 2 or 3 is the most common reason for "I pushed a fix but the live site still shows the old version." If a deploy seems to do nothing, Git Version Control shows the last deployed commit and the deploy log.

Reconnecting or rotating a service

Every integration is reached through an environment variable read at startup. Nothing is hardcoded to a specific account beyond that. To reconnect, replace, or rotate one:

1. Get the new credentials from that service's own dashboard.
2. cPanel → File Manager (turn on "Show Hidden Files") → edit `~/vetra-api/.env` and update the matching variables. This file is never in git, and deploys leave it alone.
3. cPanel → Setup Node.js App → **Restart** (or `touch ~/vetra-api/tmp/restart.txt`). A `.env` edit does nothing until the app restarts.

Service	Variables	Notes on rotating
Database	`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL`, `DB_CONNECTION_LIMIT`, `DB_QUEUE_LIMIT`	On cPanel the database is local: `DB_HOST=localhost`, `DB_SSL=false`, and `DB_NAME` / `DB_USER` carry the account prefix (`cpuser_vetra`). Moving to a different database means running the migrations against it first (every deploy runs `npm run migrate`, so an empty database gets the schema, but no rows). Copying *data* is a separate job: `npm run copy-data`, or `scripts/move-data.sh`, which was used for the Clever Cloud → cPanel move.
Cloudinary	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Rotating the key and secret does not touch any file already uploaded. Only new uploads use the new credentials. Changing the **cloud name** is different: every `res.cloudinary.com` URL already stored in the database points at the old cloud and would break.
cPanel mail	`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM`	`SMTP_HOST` must be the server's own hostname, `premium176-2.web-hosting.com`, **not** `mail.vetrahq.org`: the mail server presents Namecheap's shared `*.web-hosting.com` certificate, so `mail.vetrahq.org` fails TLS and every send fails. `SMTP_PASS` is the `noreply@vetrahq.org` mailbox password. Changing that password in cPanel breaks every send until `.env` is updated and the app restarted. `EMAIL_FROM` must be that mailbox or an alias of it. The app logs `[email] SMTP ready` or `[email] SMTP check failed` at every start.
Anthropic	`ANTHROPIC_API_KEY`, `ASSISTANT_MODEL`	The assistant errors if the key is invalid or the account is out of credit. Nothing else is affected.
Google Sign-In	`GOOGLE_CLIENT_ID`	Not secret. It is embedded in the frontend's own JS (`google-signin.js`), so seeing it in the browser is expected. Changing it means creating a new OAuth Client ID in Google Cloud Console and updating both this variable **and** `google-signin.js`, then redeploying the frontend.
JWT signing	`JWT_SECRET`, `JWT_EXPIRES_IN`	**Rotating `JWT_SECRET` signs every user out immediately**, since every previously issued token fails signature verification on its next request. That is the intended effect if you are rotating because it leaked. Do not touch it casually.
Encryption at rest	`ENCRYPTION_KEY`	Encrypts vendor bank account numbers **and** KYC identity numbers (AES-256-GCM). **Rotating this without a re-encryption step makes every already-saved payout account and identity number unreadable**, since the ciphertext was written under the old key. Every affected vendor would have to re-enter both.

Service	Variables	Notes on rotating
Paystack	`PAYSTACK_SECRET_KEY`	The same key signs webhooks, so the webhook URL keeps working across a rotation. Switching **test → live** is the big one: test payments, payouts and refunds don't exist in live mode, so cancel any test orders first (see §7), then change the key and restart. Cached transfer recipients are stored per mode and remade automatically. Saved payout accounts are unaffected.
IDCheck.ng	`IDCHECK_API_KEY`, `IDCHECK_BASE_URL` (optional, defaults to `https://api.idcheck.ng`)	Server-side only; the key must never appear in frontend JavaScript. A `sk_test_...` key returns synthetic sandbox data, and every result it produces is labelled "(IDCheck.ng sandbox, not a real check)" in the admin console, so a sandbox pass is never mistaken for a real one. Switch to a `sk_live_...` key once IDCheck has approved go-live. Rotating the key does not affect already-verified vendors.
Session behaviour	`SESSION_IDLE_TIMEOUT_MINUTES`	Server-side inactivity window, default 30 minutes. Lowering it signs idle people out sooner; raising it is a real security tradeoff, not a convenience setting.
Frontend origin	`CORS_ORIGINS`, `FRONTEND_URL`	`CORS_ORIGINS` is the allow-list of origins permitted to call the API at all. If it does not include wherever the frontend is actually served from, every request fails in the browser with a CORS error rather than a readable API error. Unset falls back to `https://vetrahq.org` and `https://www.vetrahq.org`, never `*`. `FRONTEND_URL` is used only to build links inside emails (password resets, rider invites, tracking links, the rider app link in job offers) and defaults to `https://vetrahq.org`.

Disconnecting a service

There is no off switch for most of these. Disconnecting means either removing the variables from `~/vetra-api/.env` and restarting (the feature degrades in a specific way) or closing the account on that service.

- **Cloudinary removed**: file uploads fail with a real error. Nothing else breaks, but note that existing images keep working only while the Cloudinary account itself still exists.
- **SMTP unset or failing**: every email attempt fails silently to the end user. The action it was attached to (invite, reset, order update) still completes, and the code, link, or password is logged to the app's log instead (`~/vetra-api/stderr.log` / `stdout.log` on cPanel). Grep for `[email:not-configured]`, `[email:failed]`, or `[email:fallback]`.
- **Anthropic removed**: the assistant chat errors. Nothing else is affected, and nothing currently calls it from the frontend anyway.
- **Paystack removed**: checkout stops (buyers see "Payments aren't available right now"), payout accounts can't be added, and payouts and refunds already recorded wait as failed or scheduled until it's back. Nothing is lost, but no new orders can be placed.
- **IDCheck.ng removed**: `POST /api/vendors/me/kyc/verify` returns 503 "Identity verification is not configured yet." Vendors can still upload documents; nothing can be auto-verified, so every submission would need the manual-review path. **An empty wallet** has the same practical effect without the 503: every

check fails with "The IDCheck.ng wallet is out of credit" and lands in manual review, so top the wallet up if that message appears in the console.
- **Google Sign-In removed**: "Continue with Google" fails. Email and password auth is unaffected.
- **The database cannot be disconnected without taking the whole app down.** Every route depends on it.

---

2. How the database is set up, and why

One MySQL database, 26 tables, no ORM. Every query in `backend/src/routes/*.js` is plain SQL through `mysql2/promise`. The schema lives in `backend/migrations/*.sql`, applied in filename order and tracked in `schema_migrations` (itself one of the 26). This section explains reasoning the migration files do not always spell out.

One `users` table for all four account types

Buyers, vendors, riders, and admins are rows in the same table, distinguished by `role` (`buyer` / `vendor` / `rider` / `admin`). Four tables would mean four copies of the auth, session, and password-reset logic for what is one concept. Role-specific columns sit `NULL` on rows that do not use them.

Current `users` columns:

Column	Type	Notes
`id`	`CHAR(36)`	A UUIDv7, not v4. Time-ordered on purpose, see below.
`role`	`ENUM('buyer','vendor','admin','rider')`	Which app this account belongs to.
`name`	`VARCHAR(190)`	Display name.
`first_name`, `middle_name`, `last_name`	`VARCHAR(100)`	Legal name parts, required for vendor signup. Sent to IDCheck.ng during KYC and matched against the NIN, BVN or driver's licence record. `name` remains the display field everywhere else.
`email`	`VARCHAR(190)`	Unique **per role**. The same address can exist once as a buyer and once as a vendor, never twice within one role.
`phone`, `address`	`VARCHAR`	Required by the signup routes, nullable on the column.
`state`	`VARCHAR(60)`	One of Nigeria's 36 states or the FCT, validated against a fixed list in `src/utils/nigerianStates.js`. Required for every buyer and vendor signup including Google. For a rider it is their dispatch zone: they are only offered parcels picked up in this state, and they cannot change it themselves. Admin rows have none.
`password_hash`	`VARCHAR(255)`	**bcrypt, one-way.** There is no way to look up someone's password. See §4.

Column	Type	Notes
`status`	`ENUM('active','suspended','pending','rejected','deleted')`	`pending` = vendor awaiting approval. `suspended` = admin suspension **or** the account holder's own "Deactivate", same end state either way, reversible. `rejected` = vendor application denied. `deleted` = terminal, see below.
`signup_method`	`VARCHAR(40)`	`email` or `google`.
`avatar_url`, `store_cover_url`	`VARCHAR(500)`	Cloudinary URLs, never a local path or a base64 blob.
`store_name`, `store_category`, `store_description`		Vendor-only.
`rider_vehicle`, `rider_plate`, `rider_on_duty`		Rider-only. Vehicle is `bicycle` / `motorbike` / `car` / `van`. `rider_on_duty` is the duty switch in the rider app; only on-duty riders are offered jobs.
`admin_role`	`ENUM('Super Admin','Moderator','Support')`	Admin-only. Every `requireAdminRole(...)` check in the backend gates on this, not on `role`.
`kyc_email_alerts_enabled`	`BOOLEAN`	Per-admin opt-out of the KYC alert email, default true.
`two_factor_enabled`	`BOOLEAN`	Email one-time-code sign-in. Only surfaced in the UI for vendors and admins.
`payout_bank_name`, `payout_bank_code`	`VARCHAR`	Vendor-only. `payout_bank_code` is Paystack's code, needed to re-resolve the account or eventually pay out through Paystack's Transfer API.
`payout_account_name`	`VARCHAR(190)`	Vendor-only but **not vendor-typed**. Resolved from Paystack against the bank code and NUBAN, both for the form's preview and again server-side before it is written. A vendor cannot submit a name the bank does not have on file.
`payout_account_number_enc`	`VARCHAR(255)`	The bank account number, **AES-256-GCM encrypted** (key = `ENCRYPTION_KEY`). Never stored or returned in plaintext. Every API response shows only a masked `•••• 1234`, derived by decrypting server-side and masking, never by storing a masked copy.
`last_login_at`, `last_login_ip`		The most recent successful sign-in. Full history is in `login_ip_history`.
`last_activity_at`	`DATETIME`	Updated by the auth middleware on every authenticated request. Drives the idle-session timeout.

Column	Type	Notes
`session_version`	`INT`	Incrementing this invalidates every token already issued for the account.
`password_changed_at`	`DATETIME`	Set only by a real password change. `NULL` means never changed since signup, which the UI states honestly instead of faking a date.
`created_at`	`DATETIME`	Real UTC, see the timestamp note below.

What `status = 'deleted'` means. Deleting the row is not possible without breaking every past order, report, and review that legitimately still references that account for someone else's records. Instead the delete flow scrubs personal fields (`name` → `Deleted User`, `email` → a unique `deleted-<id>@vetra.deleted` placeholder, `password_hash` → a random unusable value, phone/address/avatar/store/payout fields cleared) and, for a vendor, delists every product, all in one transaction. The row survives for referential integrity, nobody can sign into it, and it shows as "Deleted User" wherever it is still referenced. The placeholder email also frees the original address, so the same person can sign up again later.

Both self-deletion (`POST /api/auth/delete-account`) and admin deletion (`DELETE /api/admin/customers/:id` or `/vendors/:id`, Super Admin only) do the same thing.

Sessions are not just the token

A JWT alone would mean a suspension takes up to seven days to bite. It does not, because `src/middleware/auth.js` re-reads the account row on every authenticated request and checks four things: the account exists, its `status` is not `suspended` or `deleted`, the token's `sessionVersion` still matches `users.session_version`, and `last_activity_at` is within `SESSION_IDLE_TIMEOUT_MINUTES` (default 30).

Practical consequence for operators: **suspending an account, changing its role, changing its email, or resetting its password all take effect on that account's very next request**, not whenever their token expires. `session_version` is bumped by every one of those actions.

Why the IDs look like they do (UUIDv7)

Every primary key is a `CHAR(36)` UUID generated in `backend/src/utils/id.js`, but not by `crypto.randomUUID()`. That produces a UUIDv4, fully random, which InnoDB handles badly as a clustered-index primary key: random insert order means every insert can land anywhere in the B-tree, causing page splits and fragmentation as a table grows. `id.js` hand-builds a **UUIDv7** instead: the first 48 bits are the current Unix timestamp in milliseconds, the rest random, so new rows insert in roughly chronological order. The on-disk format is identical to a v4, so nothing else had to change.

Why money is stored the way it is

Every price and total (`products.price`, `orders.total`, `order_items.price_at_purchase`) is an `INT` counted in **kobo** (1 naira = 100 kobo), never a decimal or float naira value. Storing currency as a float invites the classic rounding bug where ₦0.1 + ₦0.2 is not exactly ₦0.3 in binary floating point. An integer count of the smallest

unit sidesteps it entirely. Conversion happens only at the UI boundary, in `api-client.js`'s `formatNaira()` and `nairaToKobo()`, the only two places in the codebase that multiply or divide by 100. If a raw database number looks 100x too large, that is why.

Timestamps are real UTC

The connection used to set `dateStrings: true`, which returned every `DATETIME` as a plain `"2026-09-16 21:04:32"` string with no timezone marker. A browser in Nigeria (WAT, UTC+1) parses that as *its own* local time, silently shifting every displayed timestamp an hour into the past. It showed up as a fresh event reading "1h ago" in the admin activity feed. That is fixed; the connection now returns real `Date` objects, which serialise as `...Z`-suffixed UTC and parse correctly in any timezone. If a timestamp is ever off by a suspiciously round number of hours again, a naive date string parsed as local time is the first thing to suspect.

Every table, one line each

Table	Purpose
`users`	Every buyer, vendor, rider, and admin. See above.
`products`	Vendor listings. `status`: `active` / `out_of_stock` / `removed`, all derived rather than set directly.
`orders`, `order_items`	Real orders from real checkouts. One vendor per order. `orders.delivery_fee` is kept separate from `total` (the vendor's goods), and `orders.delivery_code` is the buyer's 6-digit handover code. `order_items.status` lets a vendor drop a single line item without cancelling the whole order.
`reviews`	One per completed order (`UNIQUE KEY` on `order_id`), so "verified purchase" is enforced, not a label.
`reports`	The moderation queue: a buyer's complaint against a vendor, or a system auto-flag on a listing containing a restricted term.
`report_evidence`	A vendor's response and attachments on a report against them. Append-only; it never changes the report's status.
`activity_log`	The audit trail every admin, vendor, and system mutation writes to. Prunable by a Super Admin, see §5.
`vendor_kyc`	One row per vendor: CAC number, encrypted identity number, both document URLs, the automatic-check results (status and message for identity and for CAC), and the review status. `identity_type` is `nin`, `bvn` or `drivers_license`. Rows checked before October 2026 hold CheckID.ng results.
`admin_invites`	The invite-and-verify flow for adding an admin. A 6-digit code, bcrypt-hashed, 15-minute expiry.
`password_reset_tokens`	Single-use tokens behind the reset email link, SHA-256 hashed, 1-hour expiry.
`two_factor_codes`	6-digit sign-in codes for `two_factor_enabled` accounts. SHA-256 hashed, 10-minute expiry, 5-attempt cap per code, at most one unconsumed row per user.

Table	Purpose
`pending_email_changes`	Same shape, for an admin-initiated email change. The OTP goes to the *new* address; the admin enters it back in the console to finalise. Records which admin requested it.
`notifications`	Per-user in-app notifications for buyers and vendors: order updates, KYC decisions, account status changes, low stock.
`login_ip_history`	Every successful login's IP, for admin audit review.
`vendor_payout_account_history`	Masked account numbers only, one row per link/unlink. The encrypted real number stays on `users` and is never exposed to admins through any route.
`site_banners`	The dashboard and explore promo carousel images, admin-managed, picture-only by design.
`platform_settings`	One row (`id` always 1): the Platform Controls toggles on `admin/settings.html`, plus the delivery settings on `admin/deliveries.html` (free-delivery threshold, rider payout percent, auto-assign, buyer-protection hours, max delivery attempts).
`delivery_rates`	One row per state: intra-state and inter-state fee (kobo), ETA range, active flag. Edited by a Super Admin in the Deliveries console.
`deliveries`	One per delivery-method order: mode (`vetra` rider or vendor `self`), status, rider, pickup code, pickup and drop-off address snapshots, fee charged, rider payout and paid-at, failed attempts, proof photo, last rider position, and a timestamp per step.
`payments`	One per checkout: the cart snapshot, what Paystack charged, the processing fee, status. Orders are created from it once it's paid.
`payouts`	One per vendor or rider payout: amount, status, attempts, failure reason. Unique per job, so nothing is paid twice.
`refunds`	One per refund to a buyer's card: amount, reason, status.
`contact_messages`	Messages from the public contact form, with their status and any reply.
`delivery_events`	Append-only timeline for each delivery: who did what, with a detailed `note` for dispatch and vendors and a `public_note` for buyers and the tracking page.
`schema_migrations`	One row per applied migration filename. Created by the migration runner on first use.

Foreign keys: what cascades and what does not

Most foreign keys here are MySQL's default `RESTRICT` : you cannot delete a row something else still points to. A few are deliberately different, and it matters if you are ever deleting rows by hand.

- `activity_log.actor_user_id` and `admin_invites.invited_by_user_id` are `ON DELETE SET NULL`. Removing a user does not fail just because they have login history or once sent an invite. Those rows survive with the actor reference cleared. This used to be `RESTRICT`, which meant an admin who had ever signed in literally could not be removed from the team; every attempt failed with a generic error.
- `notifications.user_id`, `password_reset_tokens.user_id`, `two_factor_codes.user_id`, and `pending_email_changes` are `ON DELETE CASCADE`. These are per-account ephemera with no value once the account is gone.
- `order_items.order_id` and `report_evidence.report_id` are `ON DELETE CASCADE` onto their parent, as are `deliveries.order_id` and `delivery_events.delivery_id`.
- `deliveries.rider_id` and `delivery_events.actor_user_id` are `ON DELETE SET NULL`, so a removed rider leaves their delivery history behind, unattributed.
- Everything else referencing `users.id` (orders, products, reviews, reports, KYC, payout history, login history) is still `RESTRICT`, deliberately. A real order or review disappearing because a user row got deleted would be far worse than the delete simply failing.

`DELETE /api/admin/team/:id` works around this properly rather than around the constraint: it runs in a transaction that first nulls that admin's `reports.reporter_user_id`, `reports.attended_by_user_id`, and `vendor_kyc.reviewed_by_user_id` references, then deletes the row, so the records survive with the attribution cleared.

---

3. Creating a Super Admin without the console

The normal path is an existing Super Admin using `admin/settings.html`'s Add Admin flow: invite, a 6-digit code emailed to the invitee, then a temporary password emailed to them on verification. **This section is for when that is not possible**, because nobody has console access, or the last Super Admin is locked out, or you need one urgently and cannot wait on email delivery.

This still works exactly as written: an admin row needs only `id`, `role`, `name`, `email`, `password_hash`, `admin_role`, and `status`. Everything else on `users` is nullable or has a default (`session_version` defaults to 0, `two_factor_enabled` to false, `last_activity_at` stays `NULL`, which the auth middleware treats as "active now" rather than as an expired session). An admin needs no `state`, `phone`, or `address`.

The simplest place to run this is the server itself, where `~/vetra-api/.env` already points at the live database. Open cPanel → **Terminal** (or SSH in), then:

```
source ~/nodevenv/vetra-api/20/bin/activate
cd ~/vetra-api
```

(The exact `source` line is shown at the top of the app's Setup Node.js App page.) Running it from your own machine also works, with a local `backend/.env` holding the live `DB_*` values (`DB_HOST` is the server hostname

from cPanel's sidebar, not `localhost`) and your IP added under cPanel → **Remote MySQL**; remove the IP afterwards. Then, from that directory:

```
node -e "
require('dotenv').config();
const mysql = require('mysql2/promise');
const bcrypt = require('bcryptjs');
const crypto = require('crypto');

(async () => {
  const pool = await mysql.createPool({
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
    ssl: process.env.DB_SSL === 'true' ? { rejectUnauthorized: false } : undefined,
    connectionLimit: 1,
  });

  const id = crypto.randomUUID();               // a v4 UUID is fine here, see §2 on why v7 matters
for high-volume tables, which this is not
  const email = 'REPLACE_WITH_REAL_EMAIL';
  const name = 'REPLACE_WITH_REAL_NAME';
  const plainPassword = 'REPLACE_WITH_A_REAL_STRONG_PASSWORD';
  const passwordHash = await bcrypt.hash(plainPassword, 10);

  await pool.query(
    \`INSERT INTO users (id, role, name, email, password_hash, admin_role, status)
     VALUES (?, 'admin', ?, ?, ?, 'Super Admin', 'active')\`,
    [id, name, email, passwordHash]
  );

  console.log('Created Super Admin:', email, '(id:', id + ')');
  await pool.end();
})();
"
```

Replace the three `REPLACE_WITH_*` values first. Then sign in at `https://vetrahq.org/admin/login.html` with that email and password.

Do this carefully:

- The password goes into the command in **plain text**, so it lands in your shell history. Either pick one you will change immediately after first sign-in, or clear it afterwards (`history -d <line>` in bash/zsh), and do not reuse it anywhere else.
- Do not run this against the live database from a machine or network you do not trust. It is a direct, unauthenticated-from-the-app's-perspective write to `users`.
- It bypasses every protection the invite flow provides: no email verification, no proof the person actually received anything. Only do it when you are certain the address belongs to the right person.
- If the goal is to *recover* an existing Super Admin rather than create a new one, prefer resetting that account's password (below) over creating a second Super Admin you then have to remember to remove.

Resetting a locked-out account's password directly

Same connection setup. There is no way to recover a password, only to replace the hash:

```
node -e "
require('dotenv').config();
const mysql = require('mysql2/promise');
const bcrypt = require('bcryptjs');

(async () => {
  const pool = await mysql.createPool({ /* same config as above */ connectionLimit: 1 });
  const hash = await bcrypt.hash('REPLACE_WITH_A_REAL_STRONG_PASSWORD', 10);
  await pool.query(
    'UPDATE users SET password_hash = ?, password_changed_at = NOW(), session_version =
session_version + 1 WHERE email = ? AND role = ?',
    [hash, 'someone@example.com', 'admin']
  );
  await pool.end();
})();
"
```

Bumping `session_version` is the part people forget. Without it, any token already issued to that account stays valid, which defeats the point if you are doing this because the account was compromised.

---

4. Finding users and their details in the database

The admin console does not surface every raw field (payout details, the exact `admin_role` value, `signup_method`, provider KYC messages). For those, query directly. Same connection setup as §3, then plain SQL:

```
-- One user by email. Remember the same address can exist once per role.
SELECT * FROM users WHERE email = 'someone@example.com';

-- Every admin and their role
SELECT id, name, email, admin_role, status FROM users WHERE role = 'admin';

-- Suspended vendors
SELECT id, name, email, store_name FROM users WHERE role = 'vendor' AND status = 'suspended';

-- KYC submissions stuck in manual review
SELECT u.store_name, u.email, vk.status, vk.identity_provider_message, vk.cac_provider_message
FROM vendor_kyc vk JOIN users u ON u.id = vk.vendor_id
WHERE vk.status = 'manual_review';

-- A vendor's payout account, encrypted, see the decrypt note below
SELECT payout_bank_name, payout_bank_code, payout_account_number_enc, payout_account_name
FROM users WHERE id = '...';

-- Recent logins for one account
SELECT ip_address, occurred_at FROM login_ip_history
WHERE user_id = '...' ORDER BY occurred_at DESC LIMIT 20;
```

What you are actually looking at, field by field:

- `password_hash` is a bcrypt hash, starting `$2a$` or `$2b$`. **It cannot be reversed**, which is the entire point. For a locked-out user, the options are the real reset flow (an admin clicking Reset Password, which emails a link) or writing a fresh bcrypt hash as in §3. Never write a plaintext value into this column.
- `payout_account_number_enc` and `vendor_kyc.identity_number_enc` are AES-256-GCM ciphertext in the form `iv:authTag:ciphertext`, all hex. To read either, use the same `decrypt()` the app uses, with the live `ENCRYPTION_KEY`, run from `backend/` with `.env` present:

`bash node -e " require('dotenv').config(); const { decrypt } = require('./src/utils/encryption'); console.log(decrypt('PASTE_THE_ENC_VALUE_HERE')); "`

There is deliberately no API route that returns a decrypted payout number. Admin-facing routes only ever show a masked form. The one exception is `GET /api/admin/vendors/:id`, which does decrypt the KYC identity number for the admin reviewing it, since reviewing an identity document against a number you cannot see is not a review. - Every other column (`name`, `email`, `phone`, `address`, `store_name`, and so on) is plain readable text. Nothing else in `users` is hashed or encrypted.

---

5. Pruning the activity log

`admin/activity.html` has a "Clear All" button and a per-entry "✕". Both are Super Admin only and both are real deletes against `activity_log`.

- **This is irreversible.** There is no soft delete, no trash, nothing to restore from inside the app. Export first if you might want it: `SELECT * FROM activity_log` through direct database access.
- **Clearing the log leaves no trace of itself, by design.** Neither route writes a fresh row afterwards. Clearing the log is meant to actually clear it, not leave a new entry behind every time.

---

6. Riders and deliveries

The delivery system's full design (statuses, codes, fees, the dispatch console, every endpoint) is in `DELIVERY_SYSTEM.md`. This is the operator's short version.

Onboarding a rider. Riders never sign up themselves.

1. A Moderator or Super Admin opens Admin → **Deliveries** → **Riders** → **Add rider**, and enters name, email, phone, state, vehicle (bicycle, motorbike, car or van) and plate. Support admins can see riders but not add them.
2. The rider gets an email from `noreply@vetrahq.org` with a link to set their password, valid for **72 hours**. **Resend invite** on their row sends a fresh one.
3. They sign in at `https://vetrahq.org/rider/` on their phone and switch **On duty**.

The **state** is their dispatch zone: they are only offered parcels picked up from vendors in that state. You set the starting one; after that riders change it themselves from the **Delivery area** picker in the rider app, which is locked while they hold any job or offer. With auto-assign on (the default), each parcel marked ready goes to the on-duty rider in the vendor's state with the fewest live jobs, at most 5 each.

Day to day, from the Riders tab: edit details, **Suspend** (signs them out at once and returns any job they have not collected to the queue), **Reactivate**, and **Mark paid** (Super Admin only). Riders are paid automatically by Paystack transfer the moment they enter the buyer's code (§7); Mark paid only covers jobs from before online payments, recording a bank transfer you made outside the app.

Settings (Super Admin, Deliveries page): per-state fees, the rider payout percent (80% of the zone rate by default), the free-delivery threshold, auto-assign on or off, the buyer-protection window before escrow auto-releases (48 hours by default), and max delivery attempts before a parcel is returned (3 by default).

Locked delivery codes. Five wrong guesses at a buyer's 6-digit delivery code locks the order. Dispatch unlocks it from the Deliveries console after confirming with the buyer by phone.

Looking things up directly:

```
-- Riders, their zone and duty status
SELECT id, name, email, phone, state, rider_vehicle, rider_plate, rider_on_duty, status
FROM users WHERE role = 'rider';

-- A parcel's full timeline, by order tracking code
SELECT e.created_at, e.event, e.actor_role, e.note
FROM delivery_events e
JOIN deliveries d ON d.id = e.delivery_id
JOIN orders o ON o.id = d.order_id
WHERE o.tracking_code = 'VTA-XXXXXXXX'
ORDER BY e.created_at;

-- Payouts that haven't gone through, newest first
SELECT p.kind, u.name, p.amount / 100 AS naira, p.status, p.attempts, p.failure_reason
FROM payouts p JOIN users u ON u.id = p.user_id
WHERE p.status <> 'success'
ORDER BY p.created_at DESC;
```

---

7. Payments, payouts and refunds

Full design in `PAYMENTS.md`. Everything here is on **Admin** → **Payments**, which only Super Admins can see or open.

The rules in one line each. Buyers pay goods + delivery + a processing fee that covers Paystack. Riders are paid the moment the buyer's delivery code is entered. Vendors are paid goods minus the commission when escrow releases. Cancellations and returns refund the buyer's card automatically.

The two settings: - **Commission %**: taken from each sale's goods total. Applies only to orders placed after you change it. - **Automatic payouts**: the master switch. **Off by default.** While off, payouts are recorded as "Scheduled" and nothing leaves the Paystack balance. Turning it on sends everything waiting straight away.

Going live, in order: 1. In the Paystack dashboard: business verified, Transfers enabled, **transfer OTP turned off** (Settings → Preferences), webhook URL set to `https://api.vetrahq.org/api/payments/webhook`. 2. Test with the `sk_test_` key: place an order with a Paystack test card, deliver it with a rider, confirm it as the buyer, cancel another. Check each step on the Payments page. 3. **Test orders are real orders in the live database.** Cancel them from the vendor side (they're refunded in test mode) before switching keys, so they don't sit in vendors' lists. 4. Put the `sk_live_` key in `~/vetra-api/.env` as `PAYSTACK_SECRET_KEY`, restart, then turn **Automatic payouts** on. 5. Make sure collected payments stay in the Paystack balance (or top it up): that's what payouts are paid from. Confirm the settlement setup with Paystack.

Day to day: - **Failed payouts** (Payouts tab → Failed) retry by themselves every 30 minutes, up to 5 times. The reason is on the row: "insufficient balance" means top up the Paystack balance; an OTP message means transfer OTP got turned back on. **Retry** sends one straight away. - **No bank account**: the vendor or rider hasn't added one. It sends by itself the moment they do; nudge them. - **Refunds for disputes**: Refunds tab, enter the order ID, an amount (empty means everything) and a reason. A partial refund comes out of the vendor's payout. Not possible once the vendor has been paid for that order. - **A failed refund is never retried automatically.** It may have gone through on Paystack's side even though VETRA didn't hear back. Check the payment in the Paystack dashboard first, then **Retry** only if it isn't there.

Contact messages. What visitors send from the Contact page lands in **Admin → Messages** (every admin role can see it; the dashboard shows how many are new, and each new one sends admins an in-app notification). **Reply** emails the sender from `noreply@vetrahq.org` and marks it replied; if the email can't be sent, nothing is marked, so try again once mail is working. **Close** when it's dealt with. The sender gets an automatic "we got your message" email when they send it.

---

8. Operational gotchas worth knowing

Connection limits. On cPanel the database and the Node app share the hosting plan's **Entry Processes** cap (Resource Usage page; 20 on Namecheap's standard plan), which also covers the delivery cron job. `src/db.js` defaults `DB_CONNECTION_LIMIT` to 20, so `~/vetra-api/.env` sets it to **10** to leave headroom. `ER_USER_LIMIT_REACHED`, or requests hanging under load, means lower it and check Resource Usage for entry-process faults. A one-off script opens its own pool, so pass `connectionLimit: 1` to its `createPool`, as the examples above do.

Applying a schema change. Write a **new numbered migration file** in `backend/migrations/` (plain additive SQL, no down-migration) and deploy it: the cPanel deploy (`.cpanel.yml`) runs `npm run migrate` every time, and the runner skips anything already recorded in `schema_migrations`. Do not edit an already-applied file; the runner tracks filenames, not contents, so an edit is silently skipped. A migration that fails stops the deploy before the restart, so the running app keeps the old code instead of running against a half-migrated schema. The deploy log in Git Version Control shows the error.

Email failures are silent to whoever triggered them, deliberately. An admin clicking Reset Password always sees "Reset link sent", whether or not delivery succeeded, because surfacing a delivery failure leaks information to someone who may not need it. The real signal is in the app's log (`~/vetra-api/stderr.log` and `stdout.log`): `[email] SMTP ready` or `[email] SMTP check failed` at every start, `[email:failed]` when the mail server refused a send (reason included), and `[email:not-configured]` when the `SMTP_*` variables are missing. Each failure line includes the actual code, link, or password as a fallback, so nothing is unrecoverable just because the email did not land. `Invalid login` means `SMTP_PASS` no longer matches the mailbox password. cPanel → **Track Delivery** shows what happened to each message after the server accepted it, and **Email Deliverability** should show SPF and DKIM green. Namecheap shared plans cap outgoing mail per hour, which only matters at real volume.

Rate limits. `src/middleware/rateLimit.js` defines eleven per-IP limiters: signin 10 per 15 min, admin-signin 5 per 15 min, Google auth 10 per 15 min, signup 20 per hour, password-reset redemption 10 per 15 min, assistant chat 20 per 15 min, 2FA verify 20 per 15 min, 2FA resend 5 per 15 min, uploads 30 per 15 min, public delivery endpoints (fee quote, tracking lookup) 120 per 15 min, and pickup/delivery code entry 20 per 15 min. A twelfth, KYC verification, is keyed per vendor account rather than per IP: 5 attempts an hour, because every attempt is a paid IDCheck.ng lookup. If a legitimate user is locked out, the window is short; there is no admin override, and waiting is the answer.

Real client IPs. These limiters, and `login_ip_history`, depend on `app.set("trust proxy", "loopback, linklocal, uniquelocal")` in `src/app.js`. That preset trusts any number of hops through the standard private ranges and stops at the first public address. On cPanel the request reaches Node through Apache and Passenger on the same machine (a loopback hop), so the first public address is the visitor's. If Admin → Customers → login IP history ever shows `127.0.0.1` for new sign-ins, the proxy chain has changed and this setting needs another look. The preset replaced a hardcoded `1`, which stopped one hop too early on Render: rows written before that fix hold a private `10.x.x.x` address and are useless, not data to investigate.

User-controlled text is escaped at write time, not render time. `activity_log.message`, `notifications.message`, `reports.reason`, and `report_evidence.response_text` are all rendered with `innerHTML` by frontend JS with no escaping of their own. A vendor's store or product name, or a buyer's report reason, could otherwise contain a real `<script>` or `<img onerror=...>` payload that runs in an admin's, vendor's, or buyer's own authenticated session. Every such value is escaped by `src/utils/escapeHtml.js` immediately before the database write, including every `notify()` call site. One related field is escaped at *render* time instead, deliberately: `admin/assets/ui.js`'s activity feed escapes `actorName` client-side, because that value is never stored, it is a live `JOIN` onto `users.name` computed fresh on each read, so there is no write-time moment to escape it at. If a new route ever writes user-controlled text into one of those four columns, it needs the same write-time treatment. If a new admin view joins in more freeform user data for display, check whether it needs the render-time treatment instead.

KYC documents are not ordinary public URLs. They are uploaded to Cloudinary as `type: authenticated` and referenced by signed URLs, unlike product photos and banners which are plain public CDN links. `POST /api/vendors/me/kyc` validates that submitted document URLs are genuinely Cloudinary authenticated paths, so a vendor cannot point it at an arbitrary host. If KYC document links ever stop loading in the admin console, a Cloudinary credential or cloud-name change is the first thing to check.

The delivery sweep is a cron job, not part of the app. cPanel → Cron Jobs runs `scripts/delivery-cron.js` every 10 minutes and appends to `~/vetra-cron.log`. It releases escrow once the buyer-protection window has passed (which queues the vendor's payout), returns rider offers nobody accepted within 20 minutes to the queue, retries auto-assign for parcels waiting on a rider, gives back the stock of checkouts nobody paid for, and sends or retries payouts. If it stops (a changed Node path after a Node version change is the usual cause), nothing errors: offers just sit with one rider, escrow on delivered orders only releases when buyers tap "I received it", unpaid checkouts keep their stock reserved, and failed payouts stop retrying. A `delivery sweep {...}` line every 10 minutes in the log means it is running. The exact command is in `backend/README.md`, "Moving to cPanel", step 9.

There is no self-service "forgot password". Password resets are admin-triggered only: an admin clicks Reset Password on the customer or vendor detail page, the account holder gets the email link, and they set their own new password at `reset-password.html`. Nothing on the sign-in page starts that flow, so a user who is locked out has to contact support.